

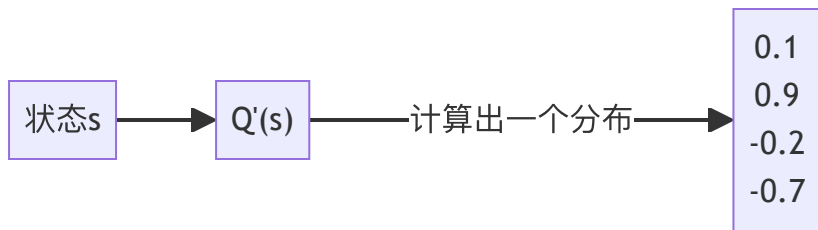
AI 学习时间

关键词：#强化学习 #灾难性遗忘 #状态叠加编码 #深度Q学习

2025-06-18 (第12课) @矩阵前端研发二组

复习

- **改进的 Q 函数**：通过状态计算出动作的 Q 值分布。

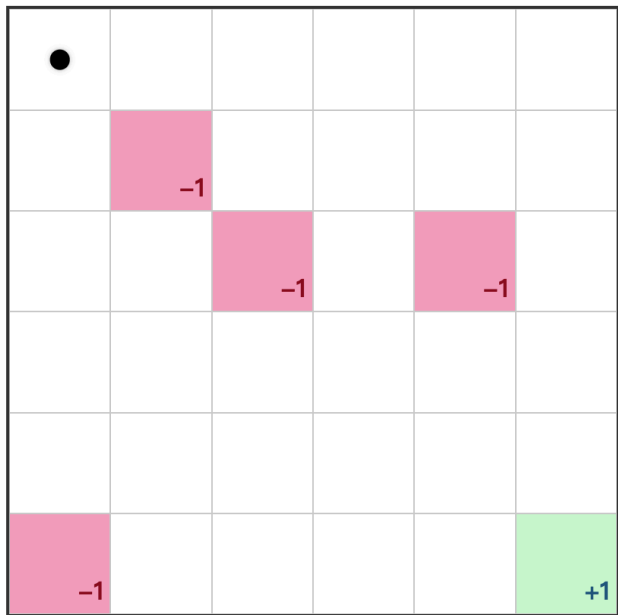


- **Q 学习**：一种强化学习方法。在学习过程中，使用 Q 函数辅助决策，通过未来收益不断调整 Q 值，最终形成一个最优策略。

$$\overbrace{Q(S_t, A_t)}^{\text{更新的 Q 值}} = \overbrace{Q(S_t, A_t)}^{\text{当前的 Q 值}} + \alpha \left[\underbrace{R_{t+1}}_{\text{奖励}} + \gamma \underbrace{\max_a Q(S_{t+1}, a)}_{\text{所有动作里的最大 Q 值}} - Q(S_t, A_t) \right]$$

GridWorld 的启示

在我们上次的例子里，GridWorld 是如下的一个格子世界。格子有三种类型，带惩罚的红色格子，带奖励的绿色格子，以及可以自由行走的普通白色格子。

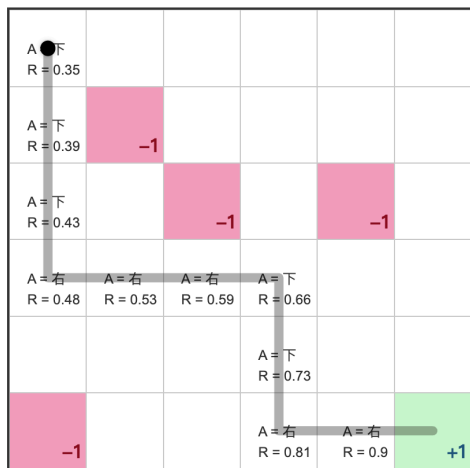


上次的问题是，训练出一个智能体，希望这个智能体在这个 GridWorld 里探索的时候能够尽可能拿到更好的奖励。

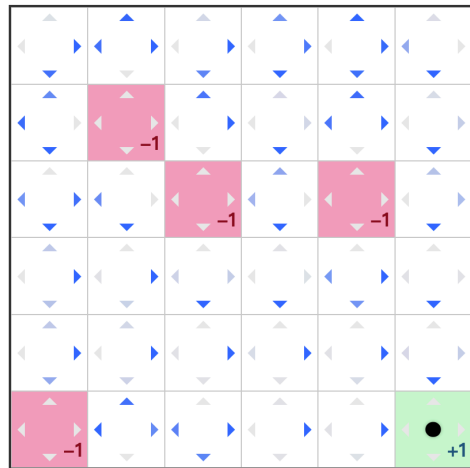
GridWorld 的启示

再次回忆我们上次用到的方法。

1. 让智能体随便走
2. 当某一条路径走到终点（假如是奖励格子）
3. 我们反向改变智能体在每一个路径格子的动作权重



通过这种方式，我们最终得到的策略可能是如下的样子。



这里需要留意的是，我们训练这个智能体的时候，使用的是一个固定的 GridWorld，也就是奖励点以及惩罚点每次都是固定的。这意味着，这个智能体只学到了“这个” GridWorld。要是我们换一个 GridWorld，奖励点和惩罚点的位置都发生变化之后，这个策略明显是行不通的，目前尝试去避免惩罚点的决策，很有可能在新的 GridWorld 里是走向惩罚点的。

GridWorld 的启示

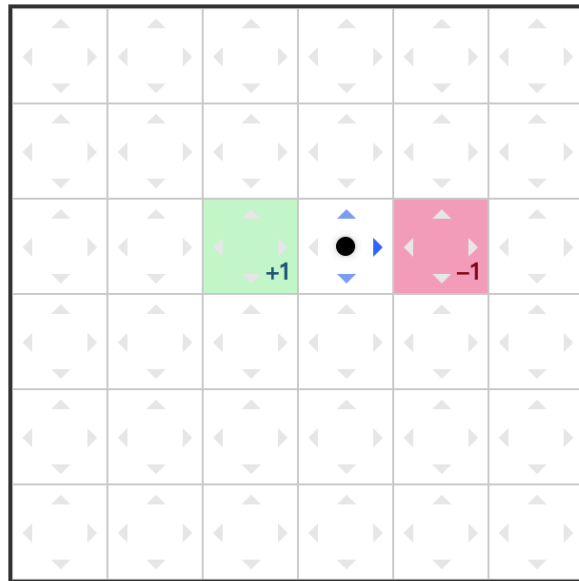
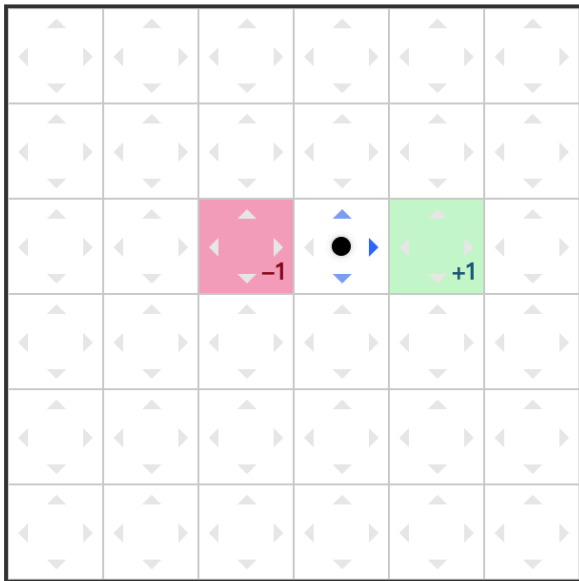
考虑一个新的问题

训练一个智能体，使其能尽量远离惩罚点，并到达奖励点获得奖励。

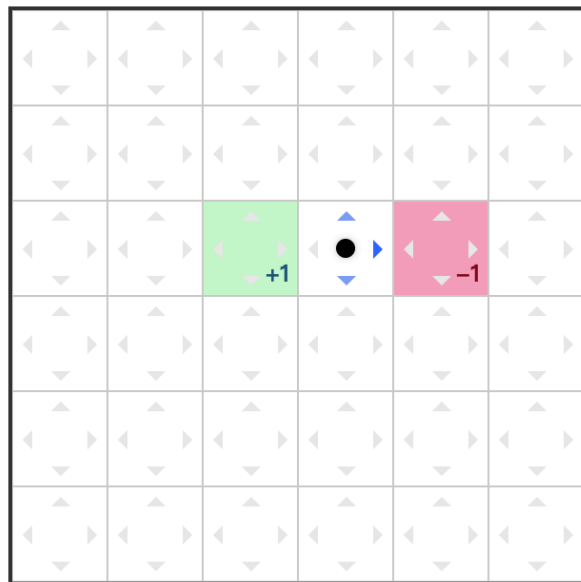
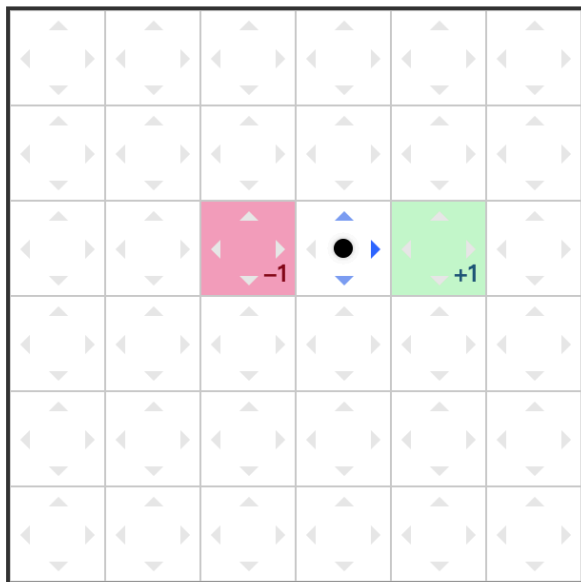
这个新问题意味着，当我们把智能体放到任意随机的 GridWorld 的时候，我们都希望它能够避开惩罚点。

灾难性遗忘

灾难性遗忘 (Catastrophic Forgetting) 在这类问题里非常普遍，我们先看看这个问题。在上面的新问题中，我们喂给智能体的是各种随机 GridWorld，下面随便给出两个例子。



灾难性遗忘

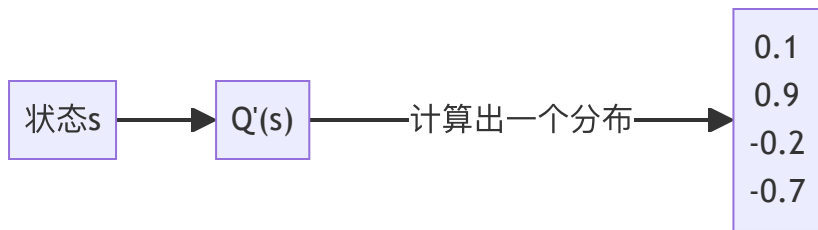


上面两个 GridWorld 的奖励点与惩罚点是刚好相反的，如果智能体在左边的 GridWorld 里学到，在 `[3, 2]` 这个位置应该往右走，那么它带着这个策略走右边的 GridWorld，将会直接走到惩罚点。此时有人可能会问，当训练到右边的 GridWorld 的时候，智能体会不会就修正自己的权重呢？但你要想的是，按照之前提到的 Q 值更新方法，会导致智能体把 Q 值修正。那么如果再给智能体左边的图，它就要陷入这种死循环里了。

位置编码

之前一直没有提到的一个细节是，我们在训练智能体的时候，GridWorld 是如何被编码的，也就是强化学习的环境，究竟是如何编码的？

考虑到改进后的 Q 函数具有以下形式：



位置编码

最容易想到的一种编码方式就是坐标编码，这种编码方式表明智能体当前的位置。比如 `s = [2, 3]` 表明智能体当前在 `[2, 3]` 这个位置。这种编码方式也是非常直观的，在这种编码方式下，执行动作并且改变状态这个过程非常直观。比如这种方式可以通过下面的方式实现。

```
1  const up = (s: State): State => {
2    const [row, col] = s
3    return [row - 1, col]
4  }
5
6  const s1 = [2, 3] // 当前智能体在 [2, 3] 这个位置
7  const s2 = up(s1) // 当前智能体在 [1, 3] 这个位置
```

这种编码方式虽然很简单很直观，但是最致命的弱点是智能体没有学到世界的所有信息，它只知道在“当前这个世界下”，在某个位置是否安全，或者在这个位置向左走，或者向右走会大概有什么结果。它并不清楚世界里，有奖励点，有惩罚点。**但我们的目标是希望智能体能够辨别奖励点，惩罚点。**只有它能够辨别，才能在另一个随机世界里也能做出正确的决策，避开惩罚点，到达奖励点。

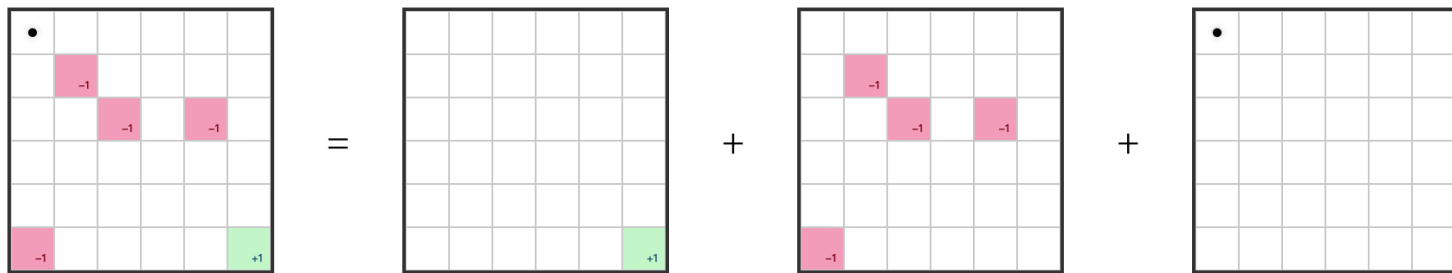
状态叠加编码

为了解决这个问题，我们需要将尽量多的环境信息编码到状态。这个涉及到训练设计的问题，不过一种很常规，并且很容易理解的办法是将各种信息分类分层，然后再叠加在一起。

比如在这个简单例子的 GridWorld 里，位置信息有三种

1. 惩罚点位置信息
2. 奖励点位置信息
3. 智能体位置信息

也就是对于最初是的 GridWorld，我们拆分出了三层信息。



状态叠加编码

这种信息转化为代码，天然是一个非常整齐的张量。

在环境中会有 `getState` 方法，通过这个方法可以获取当前环境的状态。

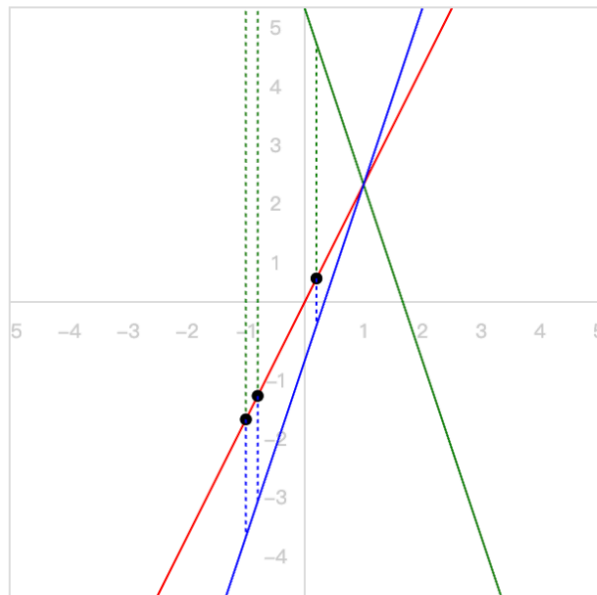
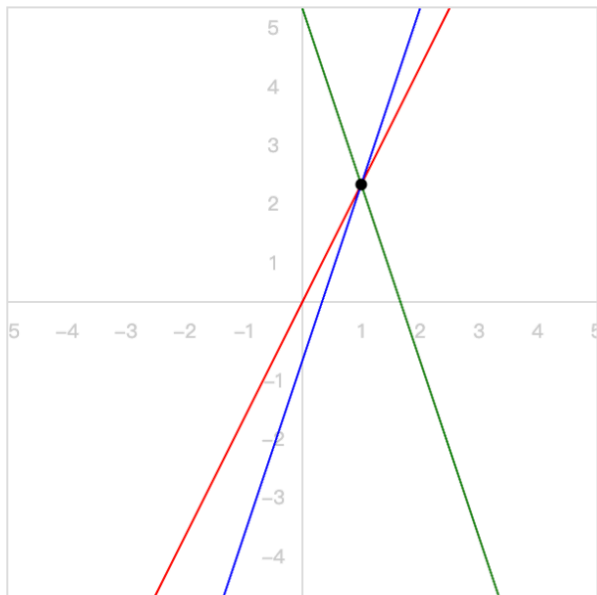
```
1 class Env {
2     getState(): State {
3         return [
4             this.rewards,
5             this.penalties,
6             this.getAgentState(),
7         ]
8     }
9 }
```

其中的 `getAgentState` 方法用于获取智能体当前位置，然后转化为这种张量格式，它所处位置的数组元素为 `1`，数组其余位置都是 `0`。这种编码方式可以让智能体在学习的过程中，“学会”这些信息之间的关联。

```
1  const state = [
2      [ // 奖励点信息
3          [0, 0, 0, 0, 0, 0],
4          [0, 0, 0, 0, 0, 0],
5          [0, 0, 0, 0, 0, 0],
6          [0, 0, 0, 0, 0, 0],
7          [0, 0, 0, 0, 0, 0],
8          [0, 0, 0, 0, 0, 1],
9      ],
10     [ // 惩罚点信息
11         [0, 0, 0, 0, 0, 0],
12         [0, 1, 0, 0, 0, 0],
13         [0, 0, 1, 0, 1, 0],
14         [0, 0, 0, 0, 0, 0],
15         [0, 0, 0, 0, 0, 0],
16         [1, 0, 0, 0, 0, 0],
17     ],
18     [ // 智能体位置
19         [1, 0, 0, 0, 0, 0],
20         [0, 0, 0, 0, 0, 0],
21         [0, 0, 0, 0, 0, 0],
22         [0, 0, 0, 0, 0, 0],
23         [0, 0, 0, 0, 0, 0],
24         [0, 0, 0, 0, 0, 0],
25     ],
26 ]
```

经验回放

回想之前讨论的训练过程，为了让学习过程更快收敛，学习的结果更为准确，其中一个技巧是**小批量学习**。



经验回放

经验回放其实与小批量的思路是一致的，就是一次迭代过程中，传入批量的经验数据。有了上面的编码之后，在一次迭代中

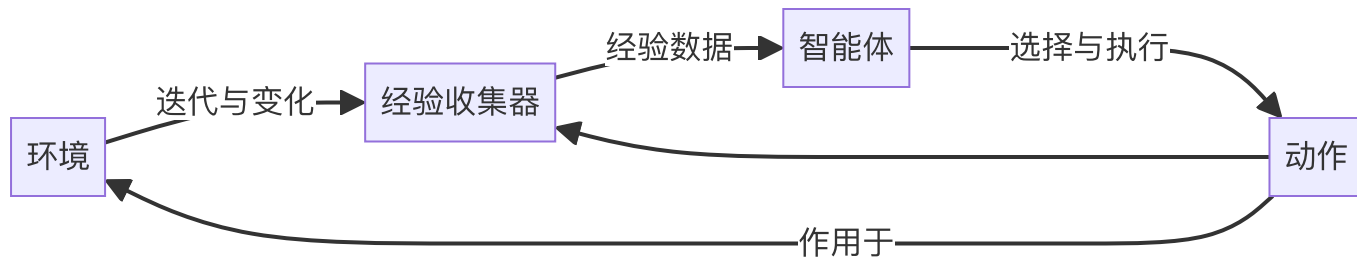
1. 获取当前状态 ``state``
2. 选择动作 ``action``
3. 执行动作
4. 环境更新状态 ``nextState``，同时反馈（奖励） ``reward``
5. 一般环境更新状态的同时会给出此次探索是否终止 ``done``

这样一条经验数据为 ``[state, action, nextState, reward, done]``。当智能体在环境中自由探索，并且没有终结之前，就能持续获得经验数据。这样在一次探索中，最后获得一组经验数据。

```
1 // 一条经验数据
2 type Experience = [State, Action, State, Reward, Done]
3 // 一次探索里得到的一组经验数据
4 type Episode = Experience[]
```

经验回放

经验回放在训练过程中起到收集经验数据，然后组织为批量数据作为训练的输入的作用。



深度 Q 学习

上次我们说到的 Q 学习，可以直接迁移到深度网络中。Q 学习的核心在于 Q 值的更新，而上次使用的是确定性的 Q 值更新方法，也就是

$$\overbrace{Q(S_t, A_t)}^{\text{更新的 Q 值}} = \overbrace{Q(S_t, A_t)}^{\text{当前的 Q 值}} + \alpha \left[\underbrace{R_{t+1}}_{\text{奖励}} + \gamma \underbrace{\max_a Q(S_{t+1}, a)}_{\text{所有动作里的最大 Q 值}} - Q(S_t, A_t) \right]$$

深度 Q 学习

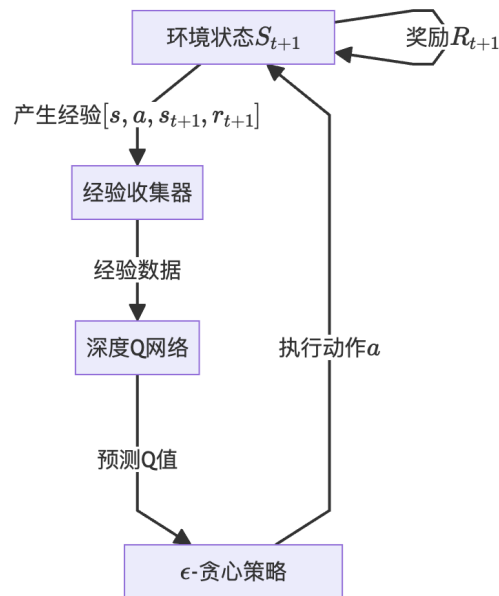
在代码中我们也是用确定性方法进行实现。

```
1  const updateQValue = (state, action, reward, nextState) => {
2    // 当前状态的 Q 值
3    const currentQ = this.qTable[state][action];
4
5    // 计算下一个状态的最大Q值
6    let maxNextQ = 0;
7    if (!done) {
8      maxNextQ = Math.max(...Object.values(this.qTable[nextState]));
9    }
10
11    // 更新公式:  $Q(s,a) = Q(s,a) + \alpha * [r + \gamma * \max_{a'} Q(s',a')] - Q(s,a)$ 
12    const newQ = currentQ + learningRate *
13      (reward + discountFactor * maxNextQ - currentQ);
14
15    // 更新 Q 表
16    this.qTable[state][action] = newQ;
17  }
```


深度 Q 学习

但经过这段时间的学习，我们应该慢慢能感受到非确定性方法所带来的魅力。在解决复杂问题的时候，非确定性方法可以不显式指定规则，通过学习的方式将“规则”内化到智能体内。所以深度 Q 学习，实际上是将 Q 值的计算交给深度神经网络进行计算，原来的`updateQValue`方法是用来存储一个巨大的 Q 值表，但现在通过神经网络计算的话，这个 Q 值表，实际上变成了网络内部的参数表示了。

结合经验回放，下面是整个深度 Q 学习的流程图。



深度 Q 学习

下面是深度 Q 学习的循环伪代码，可以与之前的进行对比。

```
1  const step = () => {
2    // 获取当前状态
3    const state = this.getState()
4    // 获取当前状态下的经验数据
5    const experience = this.getExperience()
6    // 追加到经验回放器
7    this.experienceReplay.push(experience)
8    // 获取小批量经验数据
9    const experiences = this.experienceReplay.getBatch()
10   // 使用深度 Q 网络计算 Q 值
11   const qValues = this.deepQNetwork(state, experiences)
12   // 使用  $\epsilon$ -贪心策略选择动作
13   const action = this.selectAction(qValues)
14   // 执行动作
15   this.env.takeAction(action)
16 }
```

小结

- 为了让智能体“真正学会”奖励点与惩罚点，需要
 - 通过**叠加状态的编码**方式传递环境信息
 - 通过**经验重放**来避免灾难性遗忘
- **Q 学习** 使用“巨大的 Q 值表”来维护与更新 Q 值
- **深度 Q 学习** 是使用深度网络来预测 Q 值

Q & A